**Print copy of official Microsoft Learn documentation**

Source: https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/
Exported: 2026-10-14. Source metadata date: 2025-03-10T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

# Asynchronous programming with async and await

The Task asynchronous programming (TAP) model provides a layer of abstraction over typical asynchronous coding. In this model, you write code as a sequence of statements, the same as usual. The difference is you can read your task-based code as the compiler processes each statement and before it starts processing the next statement. To accomplish this model, the compiler performs many transformations to complete each task. Some statements can initiate work and return a Task object that represents the ongoing work and the compiler must resolve these transformations. The goal of task asynchronous programming is to enable code that reads like a sequence of statements, but executes in a more complicated order. Execution is based on external resource allocation and when tasks complete.

The task asynchronous programming model is analogous to how people give instructions for processes that include asynchronous tasks. This article uses an example with instructions for making breakfast to show how the `async` and `await` keywords make it easier to reason about code that includes a series of asynchronous instructions. The instructions for making a breakfast might be provided as a list:

1. Pour a cup of coffee.
2. Heat a pan, then fry two eggs.
3. Cook three hash brown patties.
4. Toast two pieces of bread.
5. Spread butter and jam on the toast.
6. Pour a glass of orange juice.

If you have experience with cooking, you might complete these instructions **asynchronously**. You start warming the pan for eggs, then start cooking the hash browns. You put the bread in the toaster, then start cooking the eggs. At each step of the process, you start a task, and then transition to other tasks that are ready for your attention.

Cooking breakfast is a good example of asynchronous work that isn't parallel. One person (or thread) can handle all the tasks. One person can make breakfast asynchronously by starting the next task before the previous task completes. Each cooking task progresses regardless of whether someone is actively watching the process. As soon as you start warming the pan for the eggs, you can begin cooking the hash browns. After the hash browns start to cook, you can put the bread in the toaster.

For a parallel algorithm, you need multiple people who cook (or multiple threads). One person cooks the eggs, another cooks the hash browns, and so on. Each person focuses on their one specific task. Each person who is cooking (or each thread) is blocked synchronously waiting for the current task to complete: Hash browns ready to flip, bread ready to pop up in toaster, and so on.




Consider the same list of synchronous instructions written as C# code statements:

```
using System;
using System.Threading.Tasks;

namespace AsyncBreakfast
{
    // These classes are intentionally empty for the purpose of this example. They are simply marker
classes for the purpose of demonstration, contain no properties, and serve no other purpose.
    internal class HashBrown { }
    internal class Coffee { }
    internal class Egg { }
    internal class Juice { }
    internal class Toast { }

    class Program
    {
        static void Main(string[] args)
        {
            Coffee cup = PourCoffee();
            Console.WriteLine("coffee is ready");

            Egg eggs = FryEggs(2);
```

```
        Console.WriteLine("eggs are ready");

        HashBrown hashBrown = FryHashBrowns(3);
        Console.WriteLine("hash browns are ready");

        Toast toast = ToastBread(2);
        ApplyButter(toast);
        ApplyJam(toast);
        Console.WriteLine("toast is ready");

        Juice oj = PourOJ();
        Console.WriteLine("oj is ready");
        Console.WriteLine("Breakfast is ready!");
    }

    private static Juice PourOJ()
    {
        Console.WriteLine("Pouring orange juice");
        return new Juice();
    }

    private static void ApplyJam(Toast toast) =>
        Console.WriteLine("Putting jam on the toast");

    private static void ApplyButter(Toast toast) =>
        Console.WriteLine("Putting butter on the toast");

    private static Toast ToastBread(int slices)
    {
        for (int slice = 0; slice < slices; slice++)
        {
            Console.WriteLine("Putting a slice of bread in the toaster");
        }
        Console.WriteLine("Start toasting...");
        Task.Delay(3000).Wait();
        Console.WriteLine("Remove toast from toaster");

        return new Toast();
    }

    private static HashBrown FryHashBrowns(int patties)
    {
        Console.WriteLine($"putting {patties} hash brown patties in the pan");
        Console.WriteLine("cooking first side of hash browns...");
        Task.Delay(3000).Wait();
        for (int patty = 0; patty < patties; patty++)
        {
            Console.WriteLine("flipping a hash brown patty");
        }
        Console.WriteLine("cooking the second side of hash browns...");
        Task.Delay(3000).Wait();
        Console.WriteLine("Put hash browns on plate");

        return new HashBrown();
    }

    private static Egg FryEggs(int howMany)
    {
        Console.WriteLine("Warming the egg pan...");
        Task.Delay(3000).Wait();
        Console.WriteLine($"cracking {howMany} eggs");
        Console.WriteLine("cooking the eggs ...");
        Task.Delay(3000).Wait();
        Console.WriteLine("Put eggs on plate");
```

```
            return new Egg();
        }

        private static Coffee PourCoffee()
        {
            Console.WriteLine("Pouring coffee");
            return new Coffee();
        }
    }
}
```

If you interpret these instructions as a computer would, breakfast takes about 30 minutes to prepare. The duration is the sum of the individual task times. The computer blocks for each statement until all work completes, and then it proceeds to the next task statement. This approach can take significant time. In the breakfast example, the computer method creates an unsatisfying breakfast. Later tasks in the synchronous list, like toasting the bread, don't start until earlier tasks complete. Some food gets cold before the breakfast is ready to serve.

If you want the computer to execute instructions asynchronously, you must write asynchronous code. When you write client programs, you want the UI to be responsive to user input. Your application shouldn't freeze all interaction while downloading data from the web. When you write server programs, you don't want to block threads that might be serving other requests. Using synchronous code when asynchronous alternatives exist hurts your ability to scale out less expensively. You pay for blocked threads.

Successful modern apps require asynchronous code. Without language support, writing asynchronous code requires callbacks, completion events, or other means that obscure the original intent of the code. The advantage of synchronous code is the step-by-step action that makes it easy to scan and understand. Traditional asynchronous models force you to focus on the asynchronous nature of the code, not on the fundamental actions of the code.

## Don't block, await instead

The previous code highlights an unfortunate programming practice: Writing synchronous code to perform asynchronous operations. The code blocks the current thread from doing any other work. The code doesn't interrupt the thread while there are running tasks. The outcome of this model is similar to staring at the toaster after you put in the bread. You ignore any interruptions and don't start other tasks until the bread pops up. You don't take the butter and jam out of the fridge. You might miss seeing a fire starting on the stove. You want to both toast the bread and handle other concerns at the same time. The same is true with your code.

You can start by updating the code so the thread doesn't block while tasks are running. The `await` keyword provides a nonblocking way to start a task, then continue execution when the task completes. A simple asynchronous version of the breakfast code looks like the following snippet:

```
static async Task Main(string[] args)
{
    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    Egg eggs = await FryEggsAsync(2);
    Console.WriteLine("eggs are ready");
```

```
    HashBrown hashBrown = await FryHashBrownsAsync(3);
    Console.WriteLine("hash browns are ready");

    Toast toast = await ToastBreadAsync(2);
    ApplyButter(toast);
    ApplyJam(toast);
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");
    Console.WriteLine("Breakfast is ready!");
}
```

The code updates the original method bodies of `FryEggs`, `FryHashBrowns`, and `ToastBread` to return `Task<Egg>`, `Task<HashBrown>`, and `Task<Toast>` objects, respectively. The updated method names include the "Async" suffix: `FryEggsAsync`, `FryHashBrownsAsync`, and `ToastBreadAsync`. The `Main` method returns the `Task` object, although it doesn't have a `return` expression, which is by design. For more information, see Evaluation of a void-returning async function.

> Note
>
> The updated code doesn't yet take advantage of key features of asynchronous programming, which can result in shorter completion times. The code processes the tasks in roughly the same amount of time as the initial synchronous version. For the full method implementations, see the final version of the code later in this article.

Let's apply the breakfast example to the updated code. The thread doesn't block while the eggs or hash browns are cooking, but the code also doesn't start other tasks until the current work completes. You still put the bread in the toaster and stare at the toaster until the bread pops up, but you can now respond to interruptions. In a restaurant where multiple orders are placed, the cook can start a new order while another is already cooking.

In the updated code, the thread working on the breakfast isn't blocked while waiting for any started task that's unfinished. For some applications, this change is all you need. You can enable your app to support user interaction while data downloads from the web. In other scenarios, you might want to start other tasks while waiting for the previous task to complete.

## Start tasks concurrently

For most operations, you want to start several independent tasks immediately. As each task completes, you initiate other work that's ready to start. When you apply this methodology to the breakfast example, you can prepare breakfast more quickly. You also get everything ready close to the same time, so you can enjoy a hot breakfast.

The System.Threading.Tasks.Task class and related types are classes you can use to apply this style of reasoning to tasks that are in progress. This approach enables you to write code that more closely resembles the way you create breakfast in real life. You start cooking the eggs, hash browns, and toast at the same time. As each food item requires action, you turn your attention to that task, take care of the action, and then wait for something else that requires your attention.

In your code, you start a task and hold on to the Task object that represents the work. You use the `await` method on the task to delay acting on the work until the result is ready.

Apply these changes to the breakfast code. The first step is to store the tasks for operations when they start, rather than using the `await` expression:

```
Coffee cup = PourCoffee();
Console.WriteLine("Coffee is ready");

Task<Egg> eggsTask = FryEggsAsync(2);
Egg eggs = await eggsTask;
Console.WriteLine("Eggs are ready");

Task<HashBrown> hashBrownTask = FryHashBrownsAsync(3);
HashBrown hashBrown = await hashBrownTask;
Console.WriteLine("Hash browns are ready");

Task<Toast> toastTask = ToastBreadAsync(2);
Toast toast = await toastTask;
ApplyButter(toast);
ApplyJam(toast);
Console.WriteLine("Toast is ready");

Juice oj = PourOJ();
Console.WriteLine("Oj is ready");
Console.WriteLine("Breakfast is ready!");
```

These revisions don't help to get your breakfast ready any faster. The `await` expression is applied to all tasks as soon as they start. The next step is to move the `await` expressions for the hash browns and eggs to the end of the method, before you serve the breakfast:

```
Coffee cup = PourCoffee();
Console.WriteLine("Coffee is ready");

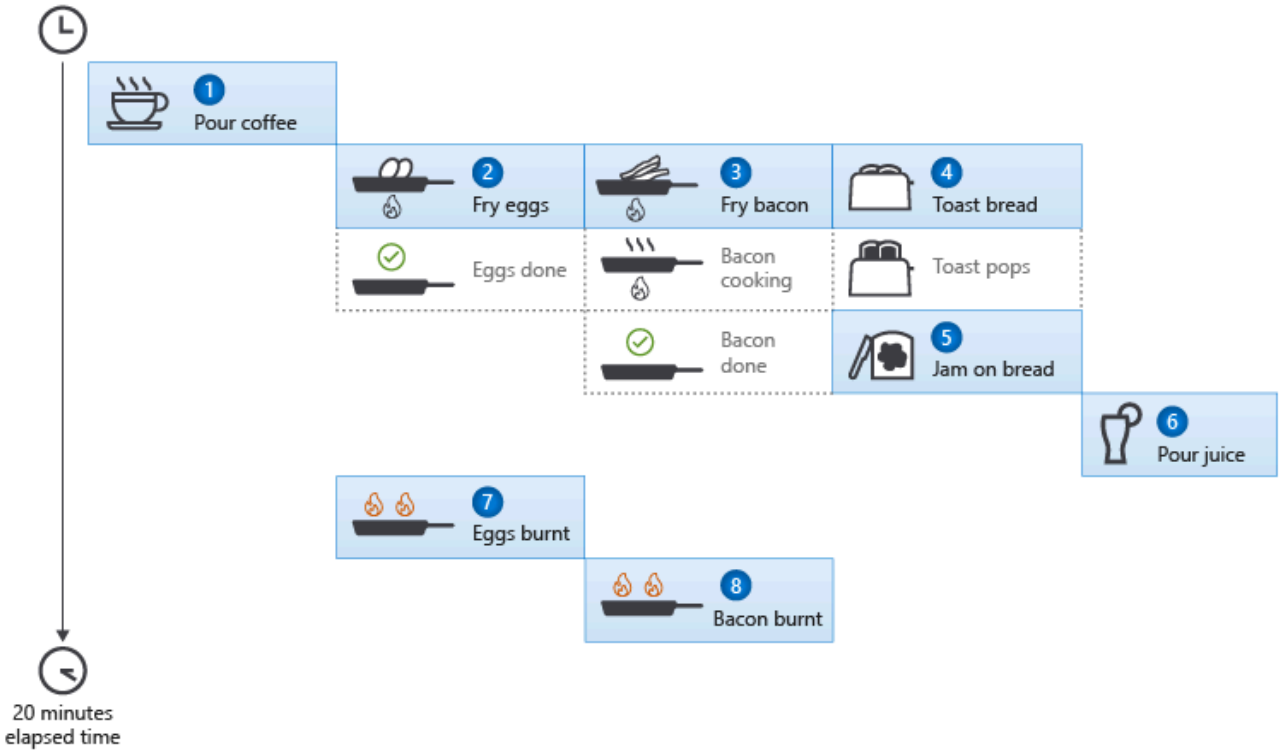
Task<Egg> eggsTask = FryEggsAsync(2);
Task<HashBrown> hashBrownTask = FryHashBrownsAsync(3);
Task<Toast> toastTask = ToastBreadAsync(2);

Toast toast = await toastTask;
ApplyButter(toast);
ApplyJam(toast);
Console.WriteLine("Toast is ready");
Juice oj = PourOJ();
Console.WriteLine("Oj is ready");

Egg eggs = await eggsTask;
Console.WriteLine("Eggs are ready");
HashBrown hashBrown = await hashBrownTask;
Console.WriteLine("Hash browns are ready");

Console.WriteLine("Breakfast is ready!");
```

You now have an asynchronously prepared breakfast that takes about 20 minutes to prepare. The total cook time is reduced because some tasks run concurrently.




The code updates improve the preparation process by reducing the cook time, but they introduce a regression by burning the eggs and hash browns. You start all the asynchronous tasks at once. You wait on each task only when you need the results. The code might be similar to program in a web application that makes requests to different microservices and then combines the results into a single page. You make all the requests immediately, and then apply the `await` expression on all those tasks and compose the web page.

## Support composition with tasks

The previous code revisions help get everything ready for breakfast at the same time, except the toast. The process of making the toast is a *composition* of an asynchronous operation (toast the bread) with synchronous operations (spread butter and jam on the toast). This example illustrates an important concept about asynchronous programming:

> Important
>
> The composition of an asynchronous operation followed by synchronous work is an asynchronous operation. Stated another way, if any portion of an operation is asynchronous, the entire operation is asynchronous.

In the previous updates, you learned how to use Task or Task<TResult> objects to hold running tasks. You wait on each task before you use its result. The next step is to create methods that represent the combination of other work. Before you serve breakfast, you want to wait on the task that represents toasting the bread before you spread the butter and jam.

You can represent this work with the following code:

```
static async Task<Toast> MakeToastWithButterAndJamAsync(int number)
{
    var toast = await ToastBreadAsync(number);
    ApplyButter(toast);
    ApplyJam(toast);

    return toast;
}
```

The `MakeToastWithButterAndJamAsync` method has the `async` modifier in its signature that signals to the compiler that the method contains an `await` expression and contains asynchronous operations. The method represents the task that toasts the bread, then spreads the butter and jam. The method returns a Task<TResult> object that represents the composition of the three operations.

The revised main block of code now looks like this:

```
static async Task Main(string[] args)
{
    Coffee cup = PourCoffee();
    Console.WriteLine("coffee is ready");

    var eggsTask = FryEggsAsync(2);
    var hashBrownTask = FryHashBrownsAsync(3);
    var toastTask = MakeToastWithButterAndJamAsync(2);

    var eggs = await eggsTask;
    Console.WriteLine("eggs are ready");

    var hashBrown = await hashBrownTask;
    Console.WriteLine("hash browns are ready");

    var toast = await toastTask;
    Console.WriteLine("toast is ready");

    Juice oj = PourOJ();
    Console.WriteLine("oj is ready");
    Console.WriteLine("Breakfast is ready!");
}
```

This code change illustrates an important technique for working with asynchronous code. You compose tasks by separating the operations into a new method that returns a task. You can choose when to wait on that task. You can start other tasks concurrently.

## Handle asynchronous exceptions

Up to this point, your code implicitly assumes all tasks complete successfully. Asynchronous methods throw exceptions, just like their synchronous counterparts. The goals for asynchronous support for exceptions and error handling are the same as for asynchronous support in general. The best practice is to write code that reads like a series of synchronous statements. Tasks throw exceptions when they can't complete successfully. The client code can catch those exceptions when the `await` expression is applied to a started task.

In the breakfast example, suppose the toaster catches fire while toasting the bread. You can simulate that problem by modifying the `ToastBreadAsync` method to match the following code:

```
private static async Task<Toast> ToastBreadAsync(int slices)
{
    for (int slice = 0; slice < slices; slice++)
    {
        Console.WriteLine("Putting a slice of bread in the toaster");
    }
    Console.WriteLine("Start toasting...");
    await Task.Delay(2000);
    Console.WriteLine("Fire! Toast is ruined!");
    throw new InvalidOperationException("The toaster is on fire");
    await Task.Delay(1000);
    Console.WriteLine("Remove toast from toaster");

    return new Toast();
}
```

> Note
>
> When you compile this code, you see a warning about unreachable code. This error is by design. After the toaster catches fire, operations don't proceed normally and the code returns an error.

After you make the code changes, run the application and check the output:

```
Pouring coffee
Coffee is ready
Warming the egg pan...
putting 3 hash brown patties in the pan
Cooking first side of hash browns...
Putting a slice of bread in the toaster
Putting a slice of bread in the toaster
Start toasting...
Fire! Toast is ruined!
Flipping a hash brown patty
Flipping a hash brown patty
Flipping a hash brown patty
Cooking the second side of hash browns...
Cracking 2 eggs
Cooking the eggs ...
Put hash browns on plate
Put eggs on plate
Eggs are ready
Hash browns are ready
Unhandled exception. System.InvalidOperationException: The toaster is on fire
   at AsyncBreakfast.Program.ToastBreadAsync(Int32 slices) in Program.cs:line 65
   at AsyncBreakfast.Program.MakeToastWithButterAndJamAsync(Int32 number) in Program.cs:line 36
   at AsyncBreakfast.Program.Main(String[] args) in Program.cs:line 24
   at AsyncBreakfast.Program.<Main>(String[] args)
```

Notice that quite a few tasks finish between the time when the toaster catches fire and the system observes the exception. When a task that runs asynchronously throws an exception, that task is **faulted**. The `Task` object holds the exception thrown in the Task.Exception property. Faulted tasks throw an exception when the `await` expression is applied to the task.

There are two important mechanisms to understand about this process:

- How an exception is stored in a faulted task

- How an exception is unpackaged and rethrown when code waits (`await`) on a faulted task

When code running asynchronously throws an exception, the exception is stored in the `Task` object. The Task.Exception property is a System.AggregateException object because more than one exception might be thrown during asynchronous work. Any exception thrown is added to the AggregateException.InnerExceptions collection. If the `Exception` property is null, a new `AggregateException` object is created and the thrown exception is the first item in the collection.

The most common scenario for a faulted task is that the `Exception` property contains exactly one exception. When your code waits on a faulted task, it rethrows the first AggregateException.InnerExceptions exception in the collection. This result is the reason why the output from the example shows an System.InvalidOperationException object rather than an `AggregateException` object. Extracting the first inner exception makes working with asynchronous methods as similar as possible to working with their synchronous counterparts. You can examine the `Exception` property in your code when your scenario might generate multiple exceptions.

> Tip
>
> The recommended practice is for any argument validation exceptions to emerge *synchronously* from task-returning methods. For more information and examples, see Exceptions in task-returning methods.

Before you continue to the next section, comment out the following two statements in your `ToastBreadAsync` method. You don't want to start another fire:

```
Console.WriteLine("Fire! Toast is ruined!");
throw new InvalidOperationException("The toaster is on fire");
```

## Apply await expressions to tasks efficiently

You can improve the series of `await` expressions at the end of the previous code by using methods of the `Task` class. One API is the WhenAll method, which returns a Task object that completes when all the tasks in its argument list are complete. The following code demonstrates this method:

```
await Task.WhenAll(eggsTask, hashBrownTask, toastTask);
Console.WriteLine("Eggs are ready");
Console.WriteLine("Hash browns are ready");
Console.WriteLine("Toast is ready");
Console.WriteLine("Breakfast is ready!");
```

Another option is to use the WhenAny method, which returns a `Task<Task>` object that completes when any of its arguments complete. You can wait on the returned task because you know the task is done. The following code shows how you can use the WhenAny method to wait on the first task to finish and then process its result. After you process the result from the completed task, you remove the completed task from the list of tasks passed to the `WhenAny` method.

```
var breakfastTasks = new List<Task> { eggsTask, hashBrownTask, toastTask };
while (breakfastTasks.Count > 0)
{
```

```
    Task finishedTask = await Task.WhenAny(breakfastTasks);
    if (finishedTask == eggsTask)
    {
        Console.WriteLine("Eggs are ready");
    }
    else if (finishedTask == hashBrownTask)
    {
        Console.WriteLine("Hash browns are ready");
    }
    else if (finishedTask == toastTask)
    {
        Console.WriteLine("Toast is ready");
    }
    await finishedTask;
    breakfastTasks.Remove(finishedTask);
}
```

Near the end of the code snippet, notice the `await finishedTask;` expression. This line is important because `Task.WhenAny` returns a `Task<Task>` - a wrapper task that contains the completed task. When you `await Task.WhenAny`, you're waiting for the wrapper task to complete, and the result is the actual task that finished first. However, to retrieve that task's result or ensure any exceptions are properly thrown, you must `await` the completed task itself (stored in `finishedTask`). Even though you know the task has finished, awaiting it again allows you to access its result or handle any exceptions that might have caused it to fault.

### Review final code

Here's what the final version of the code looks like:

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;

namespace AsyncBreakfast
{
    // These classes are intentionally empty for the purpose of this example. They are simply marker
classes for the purpose of demonstration, contain no properties, and serve no other purpose.
    internal class HashBrown { }
    internal class Coffee { }
    internal class Egg { }
    internal class Juice { }
    internal class Toast { }

    class Program
    {
        static async Task Main(string[] args)
        {
            Coffee cup = PourCoffee();
            Console.WriteLine("coffee is ready");

            var eggsTask = FryEggsAsync(2);
            var hashBrownTask = FryHashBrownsAsync(3);
            var toastTask = MakeToastWithButterAndJamAsync(2);

            var breakfastTasks = new List<Task> { eggsTask, hashBrownTask, toastTask };
            while (breakfastTasks.Count > 0)
            {
                Task finishedTask = await Task.WhenAny(breakfastTasks);
                if (finishedTask == eggsTask)
                {
                    Console.WriteLine("eggs are ready");
```

```
            }
            else if (finishedTask == hashBrownTask)
            {
                Console.WriteLine("hash browns are ready");
            }
            else if (finishedTask == toastTask)
            {
                Console.WriteLine("toast is ready");
            }
            await finishedTask;
            breakfastTasks.Remove(finishedTask);
        }

        Juice oj = PourOJ();
        Console.WriteLine("oj is ready");
        Console.WriteLine("Breakfast is ready!");
    }

    static async Task<Toast> MakeToastWithButterAndJamAsync(int number)
    {
        var toast = await ToastBreadAsync(number);
        ApplyButter(toast);
        ApplyJam(toast);

        return toast;
    }

    private static Juice PourOJ()
    {
        Console.WriteLine("Pouring orange juice");
        return new Juice();
    }

    private static void ApplyJam(Toast toast) =>
        Console.WriteLine("Putting jam on the toast");

    private static void ApplyButter(Toast toast) =>
        Console.WriteLine("Putting butter on the toast");

    private static async Task<Toast> ToastBreadAsync(int slices)
    {
        for (int slice = 0; slice < slices; slice++)
        {
            Console.WriteLine("Putting a slice of bread in the toaster");
        }
        Console.WriteLine("Start toasting...");
        await Task.Delay(3000);
        Console.WriteLine("Remove toast from toaster");

        return new Toast();
    }

    private static async Task<HashBrown> FryHashBrownsAsync(int patties)
    {
        Console.WriteLine($"putting {patties} hash brown patties in the pan");
        Console.WriteLine("cooking first side of hash browns...");
        await Task.Delay(3000);
        for (int patty = 0; patty < patties; patty++)
        {
            Console.WriteLine("flipping a hash brown patty");
        }
        Console.WriteLine("cooking the second side of hash browns...");
        await Task.Delay(3000);
        Console.WriteLine("Put hash browns on plate");
```

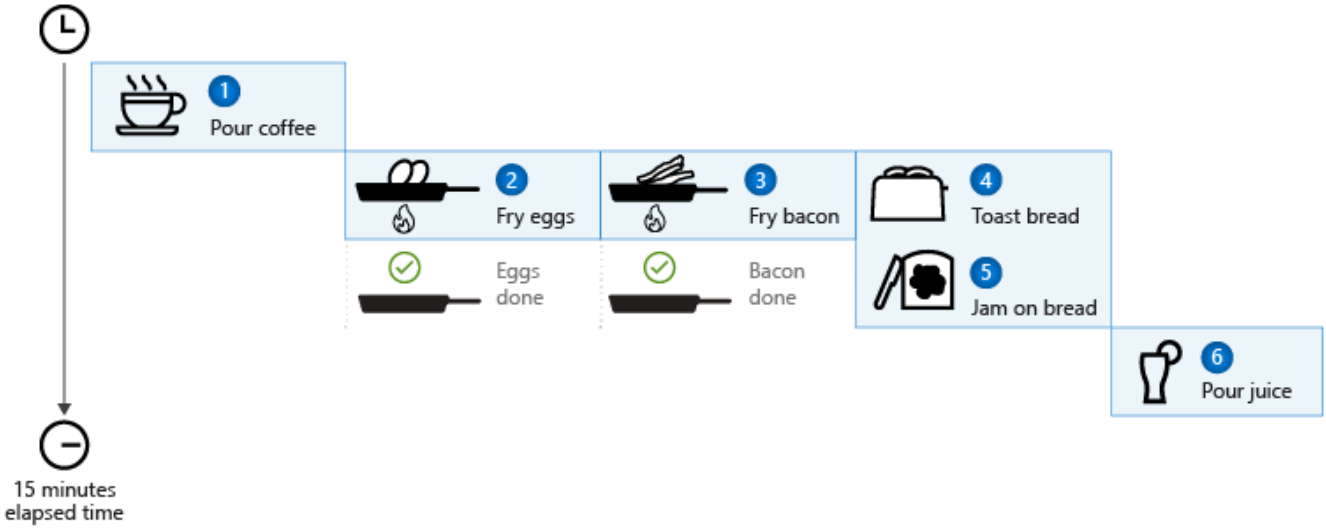
```
            return new HashBrown();
        }

        private static async Task<Egg> FryEggsAsync(int howMany)
        {
            Console.WriteLine("Warming the egg pan...");
            await Task.Delay(3000);
            Console.WriteLine($"cracking {howMany} eggs");
            Console.WriteLine("cooking the eggs ...");
            await Task.Delay(3000);
            Console.WriteLine("Put eggs on plate");

            return new Egg();
        }

        private static Coffee PourCoffee()
        {
            Console.WriteLine("Pouring coffee");
            return new Coffee();
        }
    }
}
```

The code completes the asynchronous breakfast tasks in about 15 minutes. The total time is reduced because some tasks run concurrently. The code simultaneously monitors multiple tasks and takes action only as needed.




The final code is asynchronous. It more accurately reflects how a person might cook breakfast. Compare the final code with the first code sample in the article. The core actions are still clear by reading the code. You can read the final code the same way you read the list of instructions for making a breakfast, as shown at the beginning of the article. The language features for the `async` and `await` keywords provide the translation every person makes to follow the written instructions: Start tasks as you can and don't block while waiting for tasks to complete.

## Async/await vs ContinueWith

The `async` and `await` keywords provide syntactic simplification over using Task.ContinueWith directly. While `async`/`await` and `ContinueWith` have similar semantics for handling asynchronous operations, the compiler doesn't necessarily translate `await` expressions directly into `ContinueWith` method calls. Instead, the compiler generates optimized state machine code that provides the same logical behavior. This transformation provides significant readability and maintainability benefits, especially when chaining multiple asynchronous operations.

Consider a scenario where you need to perform multiple sequential asynchronous operations. Here's how the same logic looks when implemented with `ContinueWith` compared to `async`/`await`:

### Using ContinueWith

With `ContinueWith`, each step in a sequence of asynchronous operations requires nested continuations:

```
// Using ContinueWith - demonstrates the complexity when chaining operations
static Task MakeBreakfastWithContinueWith()
{
    return StartCookingEggsAsync()
        .ContinueWith(eggsTask =>
        {
            var eggs = eggsTask.Result;
            Console.WriteLine("Eggs ready, starting bacon...");
            return StartCookingBaconAsync();
        })
        .Unwrap()
        .ContinueWith(baconTask =>
        {
            var bacon = baconTask.Result;
            Console.WriteLine("Bacon ready, starting toast...");
            return StartToastingBreadAsync();
        })
        .Unwrap()
        .ContinueWith(toastTask =>
        {
            var toast = toastTask.Result;
            Console.WriteLine("Toast ready, applying butter...");
            return ApplyButterAsync(toast);
        })
        .Unwrap()
        .ContinueWith(butteredToastTask =>
        {
            var butteredToast = butteredToastTask.Result;
            Console.WriteLine("Butter applied, applying jam...");
            return ApplyJamAsync(butteredToast);
        })
        .Unwrap()
        .ContinueWith(finalToastTask =>
        {
            var finalToast = finalToastTask.Result;
            Console.WriteLine("Breakfast completed with ContinueWith!");
        });
}
```

### Using async/await

The same sequence of operations using `async`/`await` reads much more naturally:

```
// Using async/await - much cleaner and easier to read
static async Task MakeBreakfastWithAsyncAwait()
{
    var eggs = await StartCookingEggsAsync();
    Console.WriteLine("Eggs ready, starting bacon...");

    var bacon = await StartCookingBaconAsync();
    Console.WriteLine("Bacon ready, starting toast...");

    var toast = await StartToastingBreadAsync();
    Console.WriteLine("Toast ready, applying butter...");

    var butteredToast = await ApplyButterAsync(toast);
    Console.WriteLine("Butter applied, applying jam...");

    var finalToast = await ApplyJamAsync(butteredToast);
    Console.WriteLine("Breakfast completed with async/await!");
}
```

### Why async/await is preferred

The `async`/`await` approach offers several advantages:

- **Readability**: The code reads like synchronous code, making it easier to understand the flow of operations.
- **Maintainability**: Adding or removing steps in the sequence requires minimal code changes.
- **Error handling**: Exception handling with `try`/`catch` blocks works naturally, whereas `ContinueWith` requires careful handling of faulted tasks.
- **Debugging**: The call stack and debugger experience is much better with `async`/`await`.
- **Performance**: The compiler optimizations for `async`/`await` are more sophisticated than manual `ContinueWith` chains.

The benefit becomes even more apparent as the number of chained operations increases. While a single continuation might be manageable with `ContinueWith`, sequences of 3-4 or more asynchronous operations quickly become difficult to read and maintain. This pattern, known as "monadic do-notation" in functional programming, allows you to compose multiple asynchronous operations in a sequential, readable manner.

## Next step

Explore real world scenarios for asynchronous programs